



# Vision Transformer CIFAR10 모델 Fine-tuning

JD-edu

© 2025 JD-edu. All rights reserved.

# Conda 파이썬 가상환경 설치

<https://docs.conda.io/projects/conda/en/stable/user-guide/install/index.html>

---

## 미니콘다(Miniconda)를 윈도우에 설치

리눅스의 경우 Miniforge도 사용할 수 있음

## 윈도 정상 설치방법

<https://docs.conda.io/projects/conda/en/stable/user-guide/install/windows.html>

## 미니콘다 인스톨러를 받아서 설치함

설치가 끝이 나면 다음 두 가지 실행파일이 추가됨 - **Anaconda Powershell Prompt**를 권장

- Anaconda Powershell Prompt
- Anaconda Prompt

# Conda 가상 환경 생성

Anaconda Powershell Prompt를 실행하면 다음과 같은 프롬프트가 터미널에 뜨게 됨

```
(base) PS C:\Users\conne>
```

다음 명령어를 Anaconda Powershell Prompt(이하 Powershell)에 입력하여 가상 환경을 생성. 이름을 vit\_env로 했고, 다른 이름이 가능함

```
(base) PS C:\Users\conne> conda create -n vit_env
```

Powershell에 다음 명령을 입력하여 vit\_env 가상환경을 실행

```
(base) PS C:\Users\conne> conda activate vit_env
```

Pytorch 등 필요한 라이브러리를 설치함

```
(vit_env) PS C:\Users\conne> conda install pytorch torchvision torchaudio cpuonly -c pytorch
```

Hugging Face 생태계 및 관련 도구를 설치

```
(vit_env) PS C:\Users\conne> pip install transformers datasets accelerate evaluate
```

```
(vit_env) PS C:\Users\conne> pip install scikit-learn matplotlib seaborn opencv-python
```

# Google Colab 환경 설정

01

## 소스코드 다운로드

github.com/JD-edu/deep\_learning\_class\_2의 소스코드를 다운받음 (전체 코드를 받음).

02

## 노트북 업로드

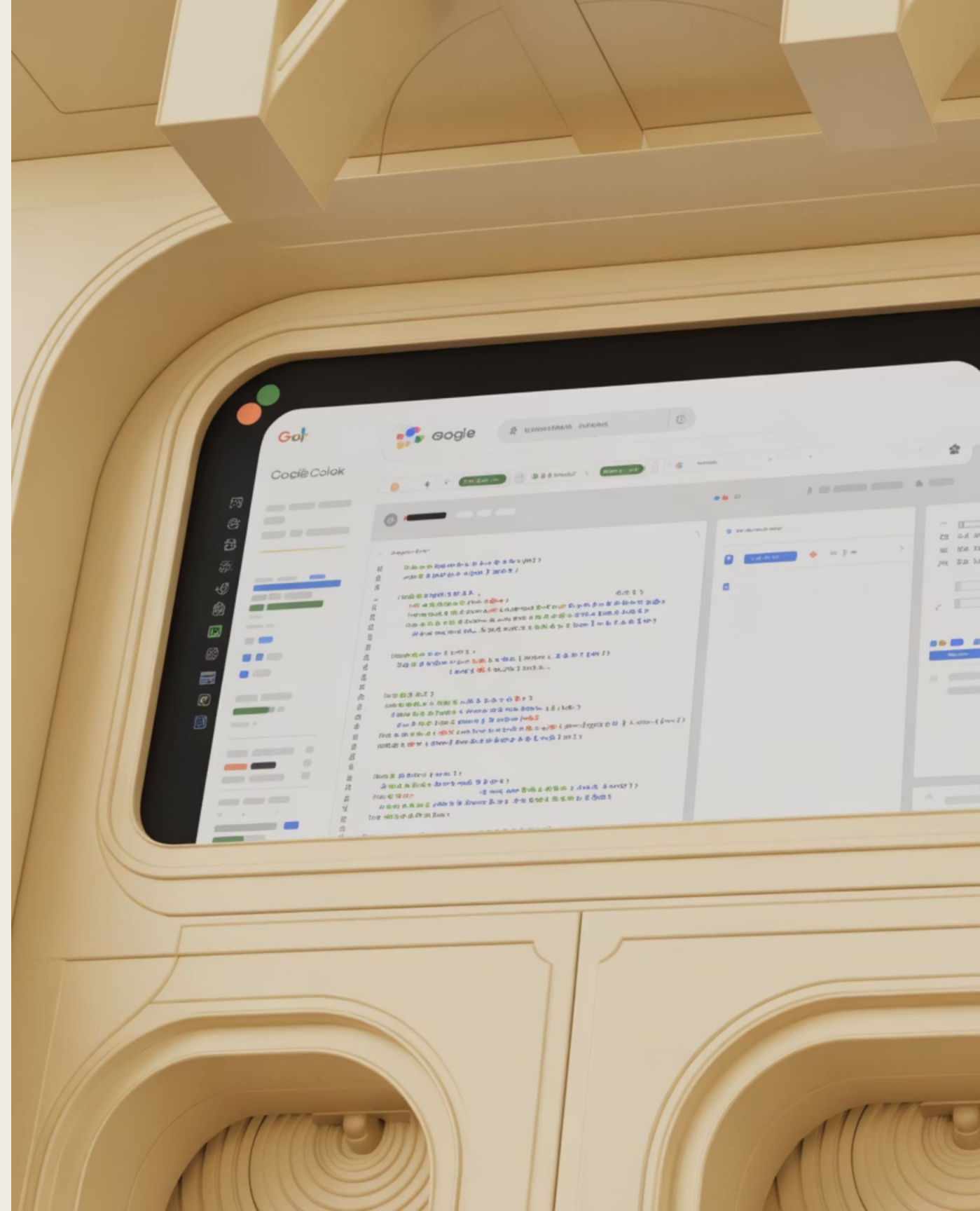
Vision\_transformer\_CIFAR10 폴더에 다음 주피터 노트북을 개인 colab에 업로드

Fine\_tuning\_ViT\_CIFAR\_10.ipynb

03

## 코드 실행

이후에는 colab의 코드블럭을 하나씩 수행하면 됨





# CIFAR10 데이터셋

## 데이터셋 상세 정보

총 이미지 수: 60,000장

(세부) 훈련 이미지 50,000장 + 테스트 이미지 10,000장

이미지 크기: 32 *times* 32 픽셀

(설명) RGB 3채널 컬러 이미지

클래스 수: 10개

(클래스 목록) airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck

클래스당 이미지 수: 6,000장

(설명) 균등 분포



# 데이터셋 로드

## Hugging Face에서 데이터셋 다운로드

다음 명령을 사용해서 데이터셋을 다운받음

`split=['train[:1000]', 'test[:200]']`를 수정해서

다운받는 데이터수량을 조절

```
train_ds, test_ds = load_dataset('cifar10',  
split=['train[:1000]', 'test[:200]'])
```

---

## 다운 받은 데이터를 분할

```
splits = train_ds.train_test_split(test_size=0.1)  
train_ds = splits['train']  
val_ds = splits['test']
```



# 다운받은 데이터 확인

OpenCV를 이용해서 데이터를 확인

```
pil_img = train_ds[231]['img']  
img_np = np.array(pil_img)  
img_bgr = cv2.cvtColor(img_np, cv2.COLOR_RGB2BGR)  
cv2.imshow('win', img_bgr)  
cv2.waitKey(2000)  
cv2.destroyAllWindows()
```

# 데이터 라벨의 실제 이름 얻기

지금까지 라벨은 숫자로 되어 있어서, 이것을 실제 라벨 이름으로 변환

## 라벨 변환 코드

```
id2label = {id:label for id, label in enumerate(train_ds.features['label'].names)}  
label2id = {label:id for id,label in id2label.items()}  
print(id2label)  
print(label2id)  
print(id2label[train_ds[231]['label']])
```

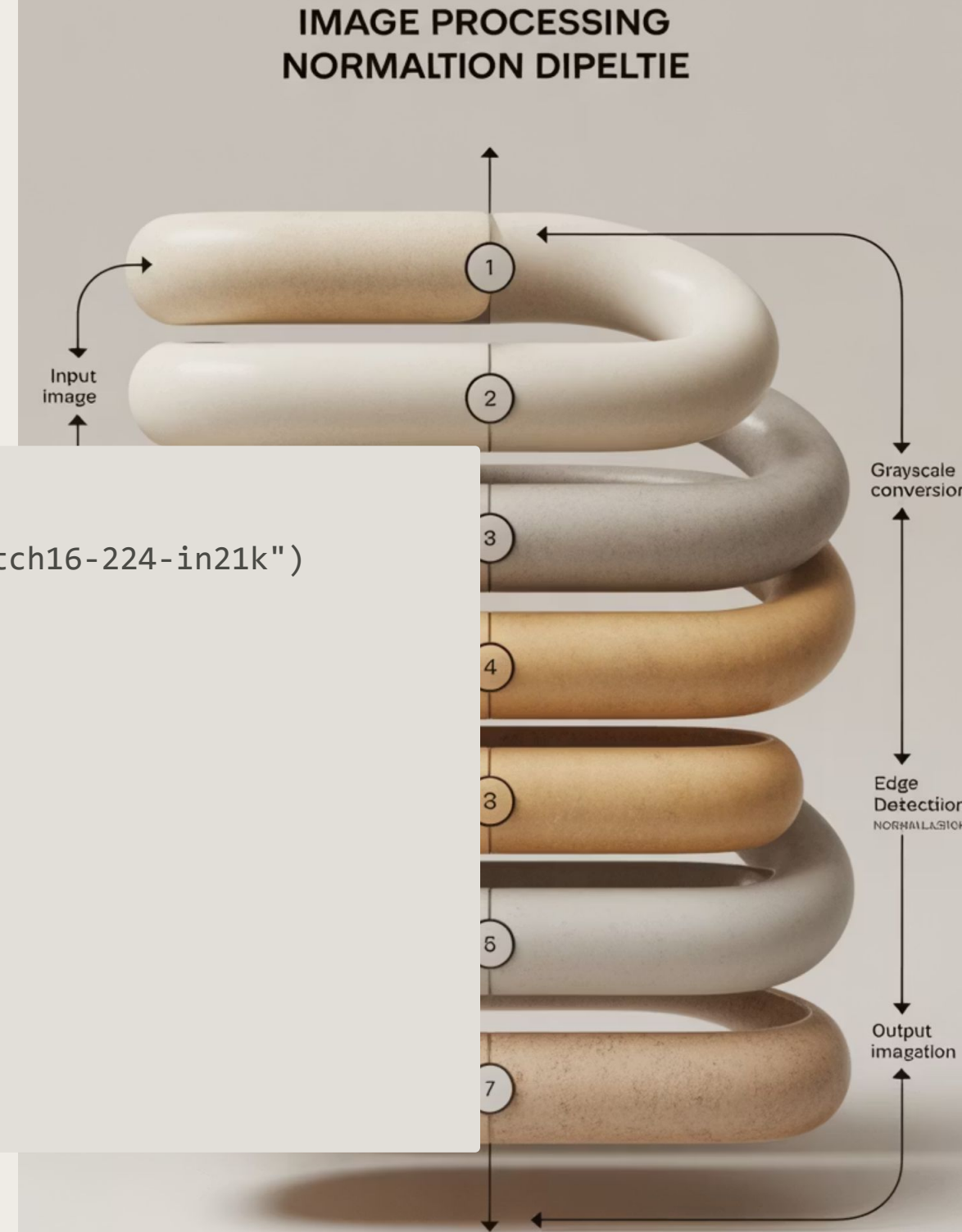


# CIFAR10 파인튜닝 진행을 위한 ViT 이미지프로세서 설정

ViT를 위한 이미지 사전 처리를 위한 ViTImageProcessor를 셋팅

```
from transformers import ViTImageProcessor
processor = ViTImageProcessor.from_pretrained("google/vit-base-patch16-224-in21k")

from torchvision.transforms import ( CenterCrop,
                                     Compose,
                                     Normalize,
                                     RandomHorizontalFlip,
                                     RandomResizedCrop,
                                     Resize,
                                     ToTensor)
```



# CIFAR10 파인튜닝 진행을 위한 ViT 이미지프로세서 설정

## 이미지 정규화 및 변환 설정

```
image_mean, image_std = processor.image_mean, processor.image_std
size = processor.size["height"]
normalize = Normalize(mean=image_mean, std=image_std)
```

## 훈련 데이터 변환

```
_train_transforms = Compose( [
    RandomResizedCrop(size),
    RandomHorizontalFlip(),
    ToTensor(),
    normalize,
])
```

# CIFAR10 파인튜닝 진행을 위한 ViT 이미지프로세서 설정

## 검증 데이터 변환

```
_val_transforms = Compose([
    Resize(size),
    CenterCrop(size),
    ToTensor(),
    normalize,
])
```

## 훈련 변환 함수

```
def train_transforms(examples):
    examples['pixel_values'] = [_train_transforms(image.convert("RGB")) for image in examples['img']]
    return examples
```

## 검증 변환 함수

```
def val_transforms(examples):
    examples['pixel_values'] = [_val_transforms(image.convert("RGB")) for image in examples['img']]
    return examples
```

# ViT CIFAR10 데이터로더 셋팅

## 데이터로더 및 배치 처리 함수 구성

```
from torch.utils.data import DataLoader
import torch

def collate_fn(examples):
    pixel_values = torch.stack([example["pixel_values"] for example in examples])
    labels = torch.tensor([example["label"] for example in examples])
    return {"pixel_values": pixel_values, "labels": labels}

train_ds.set_transform(train_transforms)
val_ds.set_transform(val_transforms)
test_ds.set_transform(val_transforms)
```

# ImageNet 기반 base모델 셋팅

```
from transformers import ViTForImageClassification
model = ViTForImageClassification.from_pretrained(
    'google/vit-base-patch16-224-in21k',
    id2label=id2label,
    label2id=label2id
)
```

```
from transformers import TrainingArguments, Trainer
metric_name = "accuracy"

args = TrainingArguments( f"test-cifar-10",
                          save_strategy="epoch",
                          eval_strategy="epoch",
                          learning_rate=2e-5,
                          per_device_train_batch_size=2,
                          per_device_eval_batch_size=1,
                          num_train_epochs=3,
                          weight_decay=0.01,
                          load_best_model_at_end=True,
                          metric_for_best_model=metric_name,
                          logging_dir='logs',
                          remove_unused_columns=False,
                          )
```





# Evaluation 진행

## 테스트 데이터 평가

```
outputs = trainer.predict(test_ds)
print(outputs.metrics)
```

## Confusion Matrix 생성

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
y_true = outputs.label_ids
y_pred = outputs.predictions.argmax(1)
labels = train_ds.features['label'].names
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(xticks_rotation=45)
```



# 트레이닝 된 모델 저장

## 최종 모델 및 프로세서 저장

```
output_dir = "./"  
print(f"Saving final model to {output_dir}...")  
trainer.model.save_pretrained(output_dir)  
processor.save_pretrained(output_dir)  
print("Model and processor saved successfully.")
```

### ✓ 모델 저장 완료

학습된 Vision Transformer 모델과 이미지 프로세서가 성공적으로 저장되었습니다.